

An aerial, top-down view of an airport tarmac. Three commercial aircraft are parked at gates, each with a yellow ground support vehicle (GPU) positioned at its tail. The tarmac is marked with yellow lines and white arrows. The background is a dark, textured surface.

Secure Wireless Aircraft Data Load

Anish Nag, Declan Creadon, Doyle Chism,
Gurumanie Singh Dhiman, Justin Rohde, Lincoln
Hattlestad

Agenda

- Problem and Context
- Requirements
- Collaboration Approach
- Design
- Prototyping and Testing
- Conclusions

Secure Wireless Aircraft
Data Load

Problem and Context

Problem and Context

Problem

- Wired (Current) Process
 - Wire into upload targets
 - Follow unique upload process for each IFE system
 - Requires custom tools
 - Manual upload is tedious, unintuitive, and non-standardized.
- Client wants a secure standard process.
 - High security risks, so...
 - Test in non-critical domains.
 - Use as prototype for mission-critical domains.

Problem and Context

Aircraft Domains

- ACD

- Aircraft Control Domain
- Mission-critical info

- AISD

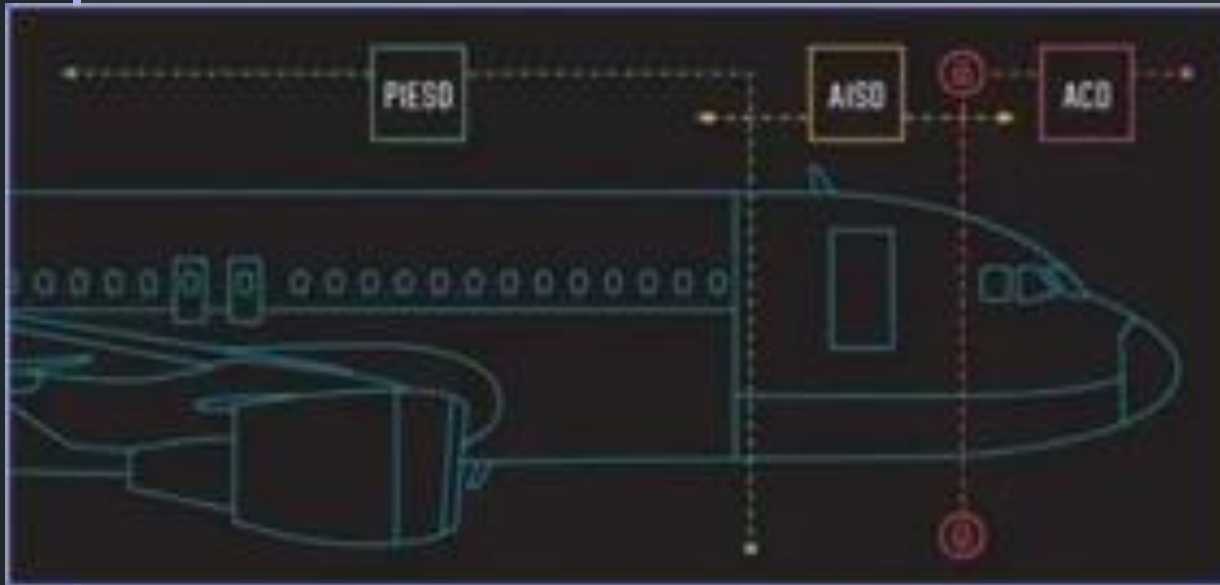
- Aircraft Information Services Domain
- Some mission-critical info

- PIESD (within our scope)

- Passenger Information and Entertainment Services Domain
- Non-mission-critical info
- All our upload targets lie in this domain

Problem and Context

Aircraft Domains



- ACD: Cabin systems, avionics, ECU, FMS, flight displays
- AISD: Internet routers, data systems, data loaders
- PIESD: IFEs, galley devices, video servers

Problem and Context

Users

- User: Aircraft Maintenance Technicians
- Use Case: Upload to unique PIESD systems using data loader in-between flight legs.
- User Needs
 - Work with tight time-lines
 - Provide passengers and pilots with fresh data
 - Upload to unique systems with singular process

- User: Aircraft Device Suppliers
- Use Case: Supply clients with hardware that follows aircraft standards and meets client needs.
- User Needs
 - Comply with aircraft technology standards
 - Comply with aircraft safety standards
 - Secure data upload method

Problem and Context

Goals

- High-level

- Improve operator ease-of-use
- Upload data of various types
- Ensure data security
- Deliver prototype for use in mission-critical domains
- Turn solution into ARINC standard (Far Future)

- Low-level

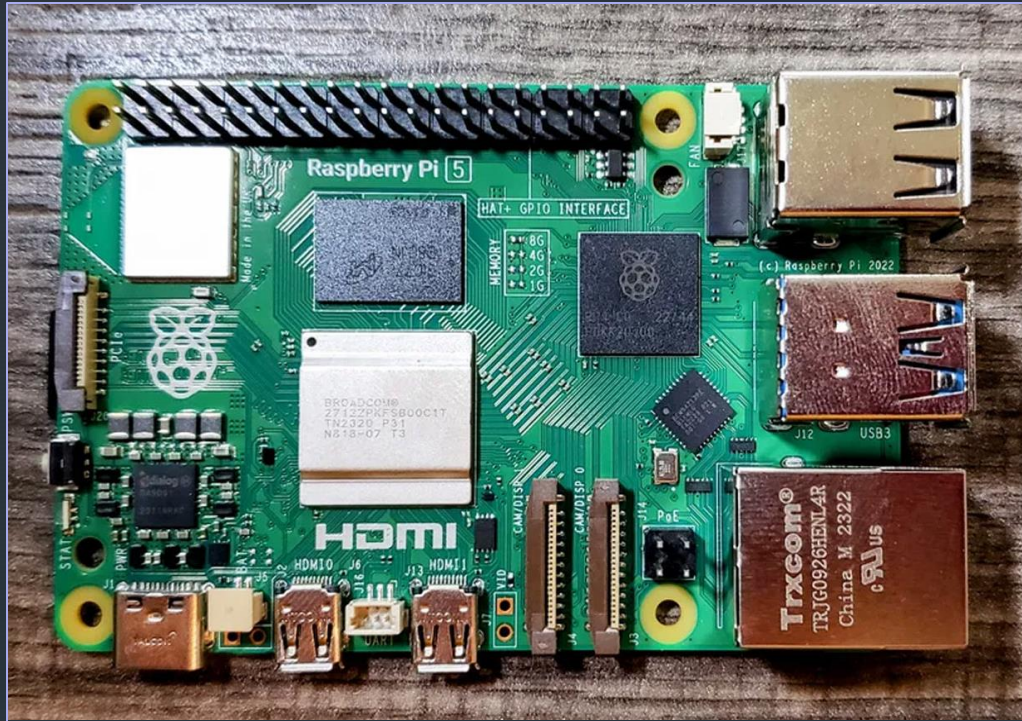
- Integrate and extend open-source ARINC libraries
- Implement ARINC (Upload, Information) operations
- Generate and manage certificates with RADIUS server
- Isolate WAP with static IP

Secure Wireless Aircraft
Data Load

Requirements

Requirements

Functional



- Wireless connectivity for data transfer between Pi's
- Linux OS
- Application layer intended for data transfer (software updates, inflight entertainment)
- Supports ARINC 615A, 665, and 645
- End-to-end data integrity

Requirements

Security



- WPA2
- EAP-TLS
- 802.1X RADIUS Authentication
- Isolated Network
- Prevent Rogue Devices

Secure Wireless Aircraft
Data Load

Collaboration Approach

Collaboration Approach

Team Structure and Scheduling



■ Team Structure

- Software Development Team: Lincoln & Doyle
- Research Team: Anish & Declan
- Network Infrastructure Team: Justin & Guru

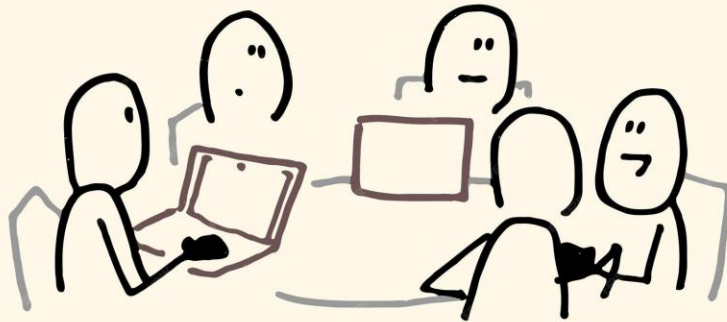
■ Scheduling

- Team meetings: Weekly
- Client advisor meetings: Weekly
- Faculty advisor meetings: Bi-weekly
- Sprint Style: Agile Scrum-like

Collaboration Approach

Key Practices

Weekly Meeting



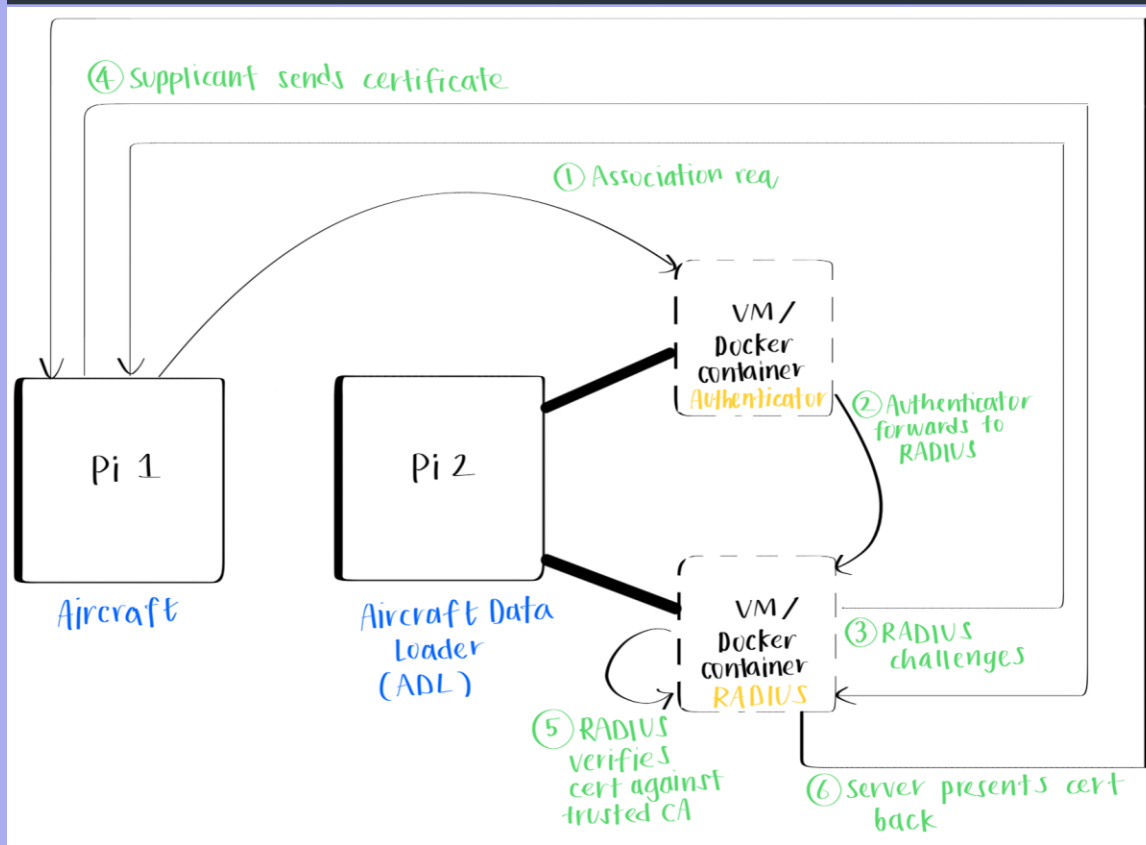
- Hour Log: Google Sheets
- Task List: Google Sheets
- Task Management: Weekly Meetings
- Pair Programming: Partners ensure work quality and progression
- Meeting Minutes: OneNote
- Research Portfolio: OneNote

Secure Wireless Aircraft
Data Load

Design

Design

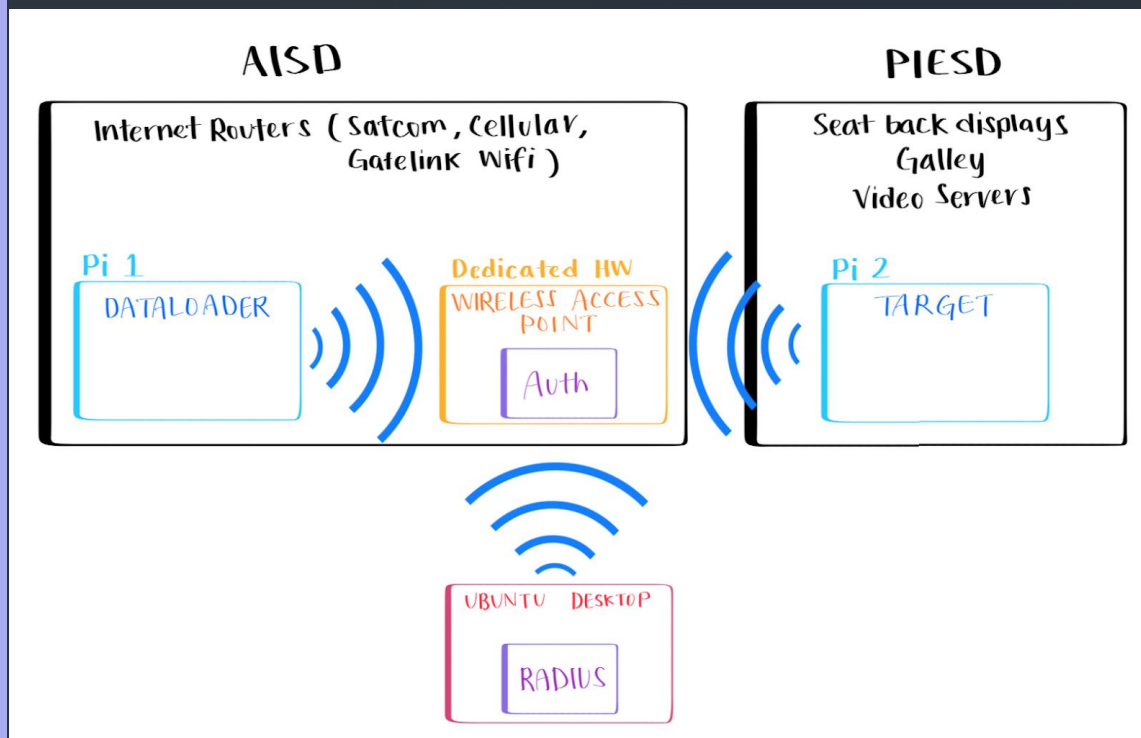
Initial System Architecture



- System Architecture
- Pi acting as Aircraft Target
- Pi acting as Aircraft Data Loader
- VM for Authentication
- VM for RADIUS

Design

Current System Architecture



- System Architecture
- Data Loader Pi
- Wireless Access Point (Cisco 140AC)
- RADIUS Server (Ubuntu 22.04 Desktop)
- Target Pi

Design Cost

Purchase	Cost	Quantity	Total
Cisco 140AC	\$90 - \$120	1	\$90 - \$120
Raspberry Pi 5 (8GB)	\$80	2	\$160
Pi Power Cables	\$0	2	\$0
Micro SD Card	\$0	2	\$0
VM (Original Design)	\$0	2	\$0
All	-	-	\$250 - \$280

Design

Engineering Design Decisions



Cisco 140AC : Fits budget constraints, supports EAP-TLS, WPA2, and WPA3.



Raspberry Pi 5 : Chosen for Linux-based flexibility and rapid prototyping capabilities.



802.1X : Industry Standard for Secure Aircraft Wireless Access



Authentication EAP Models : EAP-TTLS, EAP-Fast, PEAP, TEAP, EAP-TLS

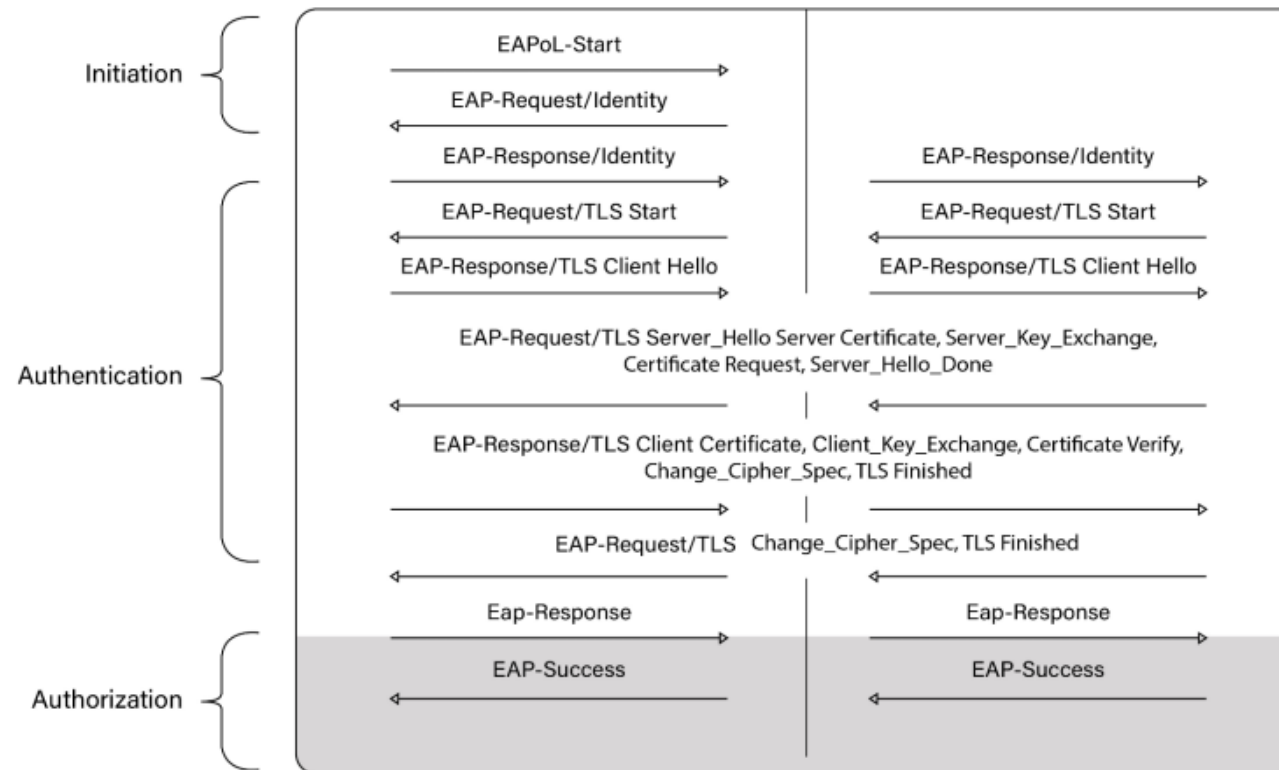
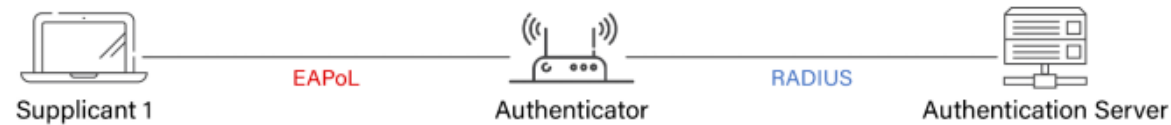


EAP-TLS: Chosen for Mutual Authentication and Fast Efficient Connections



- Device Enrollment
 - Certificates stored securely on Pi's and RADIUS server
- Authentication Workflow
 - Device connects to WAP
 - WAP forwards the auth request to RADIUS
 - RADIUS verifies identity
 - WAP grants or denies network access
- Network Segmentation
 - Entire infrastructure isolated from the internet and public view

802.1X Authentication Operation Flow



Secure Wireless Aircraft
Data Load

Prototyping and Testing

Prototyping and Testing

EAP-TLS Authentication

- Prototype 1
 - RADIUS Server: Hosted on Iowa State VM
- Outcomes
 - We were able to authenticate users with certificates.
- Drawback
 - Unable to isolate network from the internet

- Prototype 2 (Current)
 - RADIUS Server: Hosted on Ubuntu 22.04 Physical Desktop
- Outcomes
 - Successfully generated client-server certificates
 - Test users authenticated properly
- Advantage
 - Isolated from internet

Prototyping and Testing

EAP-TLS Authentication

- Authentication Testing
 - Objective: Verify that authorized users can authenticate on RADIUS
- Test Procedure:
 - Configured test user in RADIUS server
 - Sent radtest request to RADIUS
 - Valid creds => expected Access Accept
 - Invalid creds => expected Access Reject
- Outcome:
 - Correct handling of authentication attempts
 - Confirms only authorized users gain network access

Prototype and Testing

DoS Testing

- Objective:
 - Evaluate whether the RADIUS server remains stable when flooded with authentication attempts
- Bash Script:
 - Continuously sends requests using radclient
 - Sends unlimited packets until stopped
 - Tracks total authentication attempts

- Results:
 - Server did not crash
 - RADIUS continued to return Accept/Deny responses
 - Script successfully generated thousands of results
 - Log flooding occurred
 - Minor latency increase

Prototyping and Testing

Data Load

- Current Prototype

- Data Loader Pi
- Target Pi
- Home Router: AP
- ARINC 615A open-source library (C++)
- Qt based GUI

- Outcomes

- Built software utilizing CMake onto Data Loader Pi
- Qt dependency errors

- Next Steps

- Basic data transfer
- Analyze and upgrade built-in CTests
- Identify potential GUI improvements

Secure Wireless Aircraft
Data Load

Conclusions

Conclusions

Progress Summary

- While we faced some setbacks this semester, we put a good foot forward for next semester.
- So far, we have the following items done successfully:
 - RADIUS Server configured and authenticating test devices
 - Raspberry Pi's are configured for development
 - Cisco Access Point set up in an isolated environment
 - ARINC software built on the Data Loader Pi
- Progress should continue to accelerate

Conclusions

Challenges

- Library and Package issues with ARINC 615A software
- IP Lease creation problems with Access Point
- Scope misunderstanding
- Hardware choices didn't meet design
- Requirement gathering
- Client communication/approval
- Lack of codebase guidance

Conclusions

Moving Forward

- Ensure weekly client communication
- Break high-level goals into smaller, more executable tasks
- Adjust timeline to accommodate setbacks
- Key Changes
 - Communication with ARINC library maintainer
 - Scheduling: Waterfall goal calendar
 - Task Management: GitHub Issues, Waterfall goal calendar
 - Progress Tracking: Work reviews, Sheets hour log
 - Meeting Structure: Appoint rotating meeting leader

Conclusions

Next Steps

- Next steps and goals
 - Establish Data Loader Pi and Target Pi communication on isolated network
 - Discuss possible software payloads with client advisor for ideal testing
 - Make ARINC software functional
 - Test limits and add onto project timeline
 - Polish test environment documentation

Questions?



Thank You!

Secure Wireless Aircraft
Data Load

Appendix

ARINC Standards

Standard	Primary Purpose	Interface / Layer	What It Defines	Common Use
ARINC 665	Software part packaging	File structure	LSAP layout, metadata, checksums	Ensures consistent software package formatting
ARINC 615A	Data loading protocol	Ethernet/IP	Loader ↔ LRU communication, load sequence	Transfers & installs avionics software over network
ARINC 645	Onboard data management	System-level	How onboard loaders manage software	Aircraft-wide management & loading of software parts

Detailed 615A Workflow

- **615A Load Sequence**
 - **Connect** --> LRUs discover the Data Loader Pi (or vice-versa)
 - **Initialize**--> LSAP metadata validated (from ARINC 665)
 - **Transfer**--> File blocks transferred via TCP/UDP
 - **Activate**--> Target acknowledges load + restarts subsystem

EAP-TLS In Depth

- **Mutual Certificate Authentication**

- EAP-TLS uses X.509 certificates on both the client (Data Loader Pi / Target Pi) and the RADIUS server.
- During authentication, both sides validate each other's certificates, ensuring the device is trusted *before* it ever receives network access.
- This eliminates reliance on passwords or shared secrets, which can be intercepted, guessed, or reused across devices.

- **Strong Defense Against Rogue or Unauthorized Devices**

- Only devices that possess a valid client certificate signed by our RADIUS CA can successfully join the isolated aircraft network.
- A random laptop, Raspberry Pi, or malicious device cannot authenticate; even if it knows the SSID, WPA2 settings, or tries spoofing MAC addresses.
- Prevents common wireless intrusion vectors such as credential replay, dictionary attacks, or unauthorized supplicants.

- **Cryptographic Protection Through TLS**

- EAP-TLS establishes a full TLS tunnel between the device and the RADIUS server.
- This provides encryption, integrity protection, and replay protection for the entire authentication exchange.
- TLS ensures that an attacker cannot read, modify, or inject messages during authentication.

Testing Evidence

DoS Testing Script

```
#!/bin/bash

RADIUS_SERVER="10.29.160.201"
SHARED_SECRET="test"

echo "Starting RADIUS Flood Attack Simulation"
echo "Target: $RADIUS_SERVER"
echo "Users: justin, guru"
echo "Password: test"
echo "Press Ctrl+C to stop the attack"

# Counter for tracking requests
counter=0

# Cleanup function
cleanup() {
    echo ""
    echo "=== Attack Summary ==="
    echo "Total authentication requests sent: $counter"
    echo "Flood attack stopped."
    exit 0
}

# Handle Ctrl+C
trap cleanup SIGINT

echo "=== Starting in 3 seconds ==="
sleep 3

# Main flood loop
while true; do
    # Flood with user 'justin'
    echo "User-Name=justin, User-Password=test" | radclient -q $RADIUS_SERVER auth $SHARED_SECRET &

    # Flood with user 'guru'
    echo "User-Name=guru, User-Password=test" | radclient -q $RADIUS_SERVER auth $SHARED_SECRET &

    # Increment counter (2 requests per loop iteration)
    ((counter+=2))

    # Display progress every 20 requests
    if [ $((counter % 20)) -eq 0 ]; then
        echo "Requests sent: $counter"
    fi
done
```

- To run: `./radiusflood.sh`
- Next slide illustrates how the RADIUS server handles this attack

Testing Evidence

DoS Attack on RADIUS

```
(10)      [noop] = noop
(10)      } # else = noop
(10)      } # policy remove_reply_message_if_eap = noop
(10)      if (EAP-Key-Name && &reply:EAP-Session-Id) {
(10)      if (EAP-Key-Name && &reply:EAP-Session-Id) -> FALSE
(10)      } # post-auth = noop
(10) Sent Access-Accept Id 82 from 192.168.131.136:1812 to 192.168.131.1:57017 length 38
(10) Finished request
(11) Received Access-Request Id 45 from 192.168.131.1:56979 to 192.168.131.136:1812 length 62
(11) Message-Authenticator = 0x088bf7930e6bad5e5ba308fcc6a8d766 length 62
(11) User-Name = "guru"
(11) User-Password = "test"
(11) # Executing section authorize from file /etc/freeradius/3.0/sites-enabled/default
(11) authorize {
(11)   policy filter_username {
(11)     if (&User-Name) {
(11)     if (&User-Name) -> TRUE
(11)     if (&User-Name) {
(11)       if (&User-Name =~ / /) {
(11)       if (&User-Name =~ / /) -> FALSE
(11)       if (&User-Name =~ /[^\@]*@/ ) {
(11)       if (&User-Name =~ /[^\@]*@/ ) -> FALSE
(11)       if (&User-Name =~ /\.\./ ) {
(11)       if (&User-Name =~ /\.\./ ) -> FALSE
(11)       if ((&User-Name =~ /@/) && (&User-Name !~ /@(.+)\.(\.+)$/)) {
(11)       if ((&User-Name =~ /@/) && (&User-Name !~ /@(.+)\.(\.+)$/)) -> FALSE
(11)       if (&User-Name =~ /\.$/) {
(11)       if (&User-Name =~ /\.$/) -> FALSE
(11)       if (&User-Name =~ /@\.\/) {
(11)       if (&User-Name =~ /@\.\/) -> FALSE
(11)     } # if (&User-Name) = notfound
(11)   } # policy filter_username = notfound
(11) [preprocess] = ok
(11) [chap] = noop
(11) [mschap] = noop
(11) [digest] = noop
(11) suffix: Checking for suffix after "@"
(11) suffix: No '@' in User-Name = "guru", looking up realm NULL
(11) suffix: No such realm "NULL"
(11) [suffix] = noop
(11) eap: No EAP-Message, not doing EAP
(11) [eap] = noop
(11) [files] = noop
(11) [expiration] = noop
(11) [logintime] = noop
(11) pap: WARNING: No "known good" password found for the user. Not setting Auth-Type
(11) pap: WARNING: Authentication will fail unless a "known good" password is available
(11) [pap] = noop
(11) } # authorize = ok
(11) ERROR: No Auth-Type found: rejecting the user via Post-Auth-Type = Reject
```

To direct input to this VM, click inside or press Ctrl+G.

7:50 PM
12/10/2025

- This video shows our RADIUS server logs while the DoS bash script runs.
- Notice how after hundreds of authentication attempts are sent, the server remains active.

Radtest Logs

Verifies Authentication

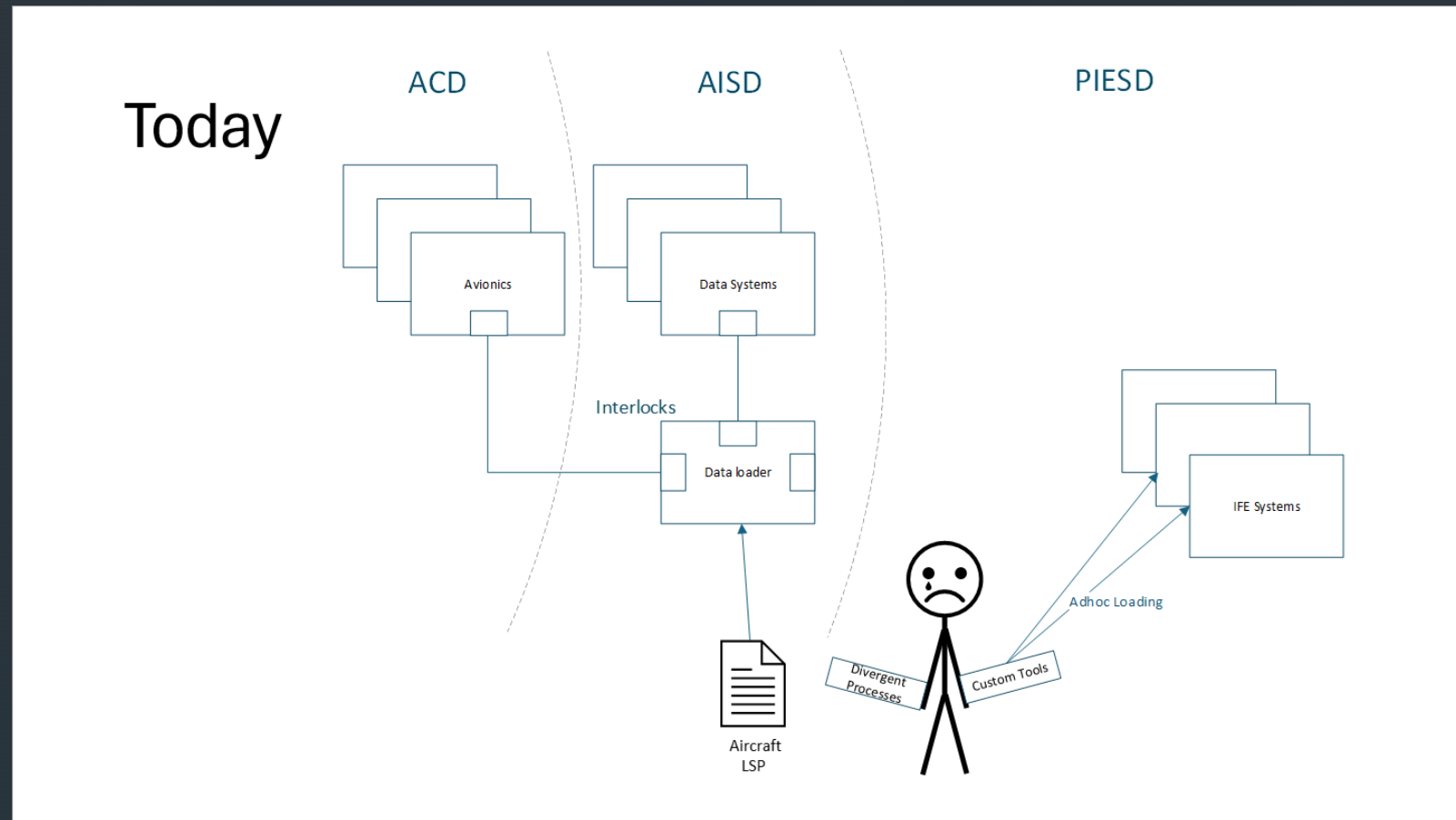
Testing from
trusted client

```
$ radtest "justin" "test" 192.168.131.136 0 test
Sent Access-Request Id 104 from 0.0.0.0:41562 to 192.168.131.136:1812 length 76
  User-Name = "justin"
  User-Password = "test"
  NAS-IP-Address = 127.0.1.1
  NAS-Port = 0
  Message-Authenticator = 0x00
  Cleartext-Password = "test"
Received Access-Accept Id 104 from 192.168.131.136:1812 to 172.19.11.83:41562 length 38
  Message-Authenticator = 0xe4fa97a953e58485d0c361333fff8bc7
```

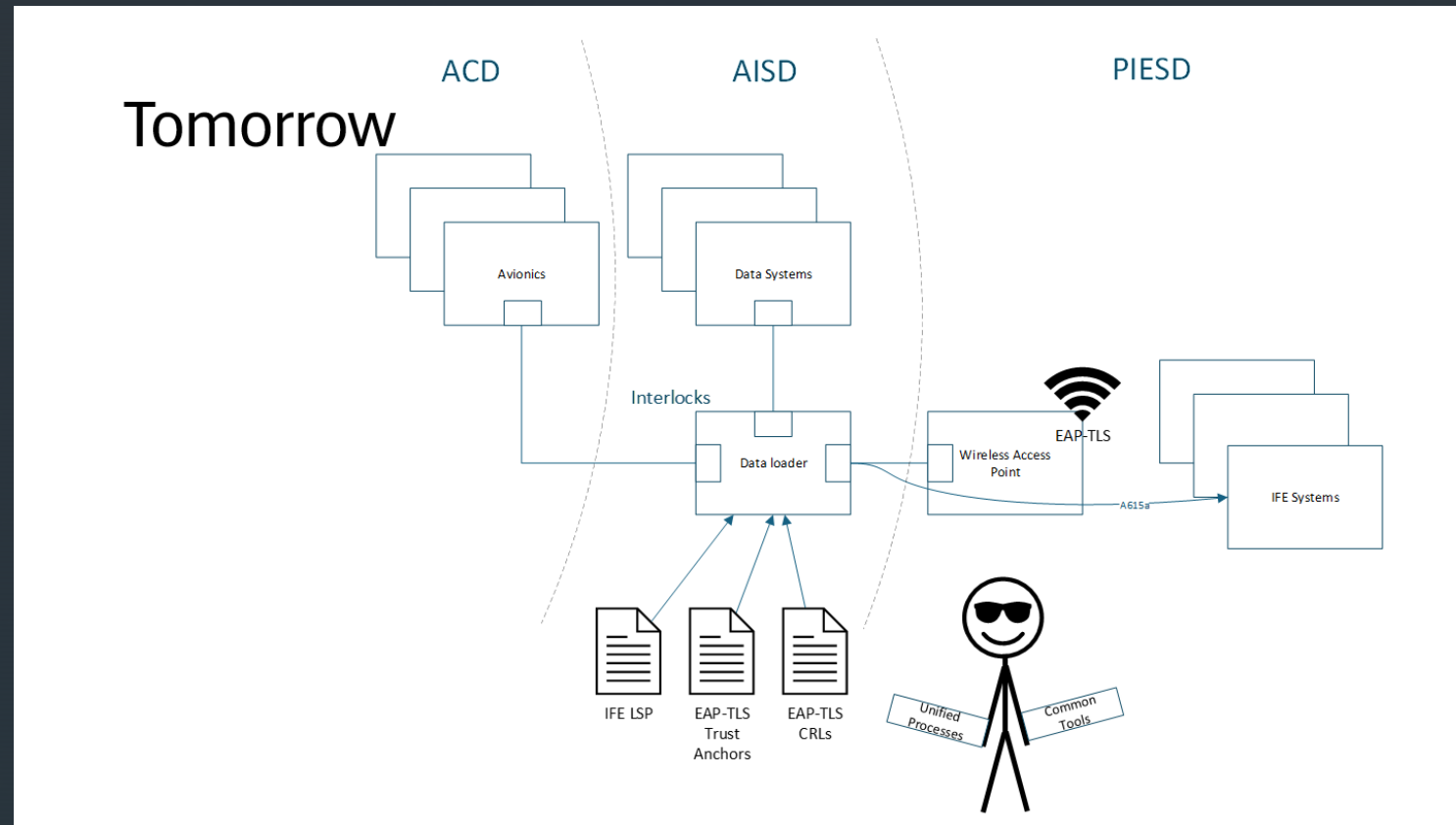
Radius Server
Logs

```
(0) suffix: Checking for suffix after '@'
(0) suffix: No '@' in User-Name = "justin", looking up realm NULL
(0) suffix: No such realm "NULL"
(0) [suffix] = noop
(0) eap: No EAP-Message, not doing EAP
(0) [eap] = noop
(0) files: users: Matched entry justin at line 21
(0) [files] = ok
(0) [expiration] = noop
(0) [logintime] = noop
(0) [pap] = updated
(0) } # authorize = updated
(0) Found Auth-Type = PAP
(0) # Executing group from file /etc/freeradius/3.0/sites-enabled/default
(0) Auth-Type PAP {
(0) pap: Login attempt with password
(0) pap: Comparing with "known good" Cleartext-Password
(0) pap: User authenticated successfully
(0) [pap] = ok
(0) } # Auth-Type PAP = ok
(0) # Executing section post-auth from file /etc/freeradius/3.0/sites-enabled/default
(0) post-auth {
(0)   if (session-state:User-Name && reply:User-Name && request:User-Name && (reply:User-Name == request:User-Name)) {
(0)     if (session-state:User-Name && reply:User-Name && request:User-Name && (reply:User-Name == request:User-Name)) -> FALSE
(0)     update {
(0)       No attributes updated for RHS &session-state:
(0)     } # update = noop
(0)     [exec] = noop
(0)     policy remove_reply_message_if_eap {
(0)       if (&reply:EAP-Message && &reply:Reply-Message) {
(0)         if (&reply:EAP-Message && &reply:Reply-Message) -> FALSE
(0)       } else {
(0)         [noop] = noop
(0)       } # else = noop
(0)     } # policy remove_reply_message_if_eap = noop
(0)     if (EAP-Key-Name && &reply:EAP-Session-Id) {
(0)       if (EAP-Key-Name && &reply:EAP-Session-Id) -> FALSE
(0)     } # post-auth = noop
(0) Sent Access-Accept Id 104 from 192.168.131.136:1812 to 192.168.131.1:55538 length 38
(0) Finished request
(0) Taking up in 4.9 seconds.
(0) Cleaning up request packet ID 104 with timestamp +5 due to cleanup_delay was reached
Ready to process requests
```

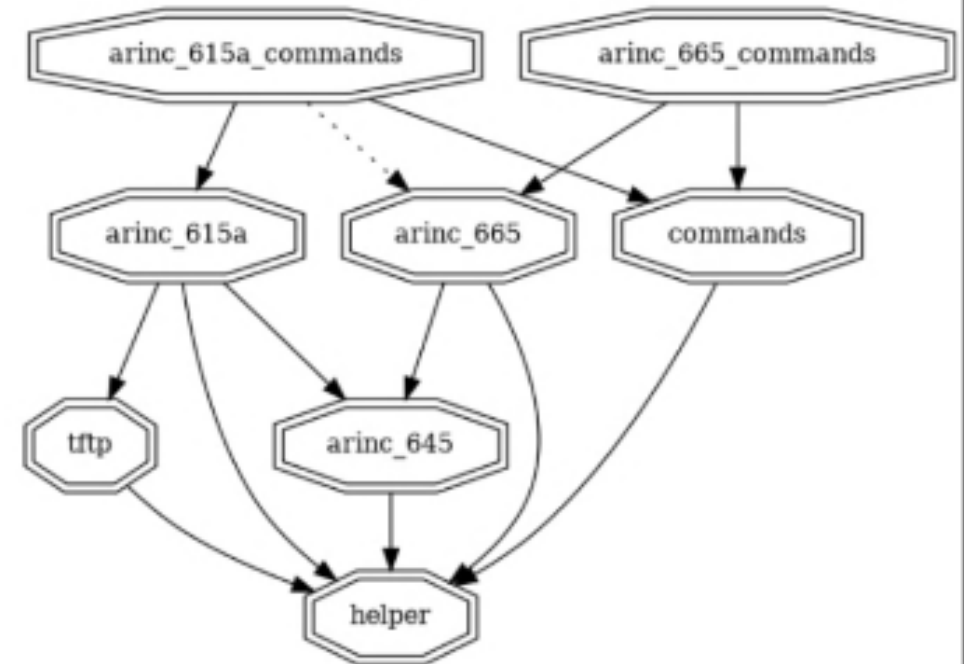
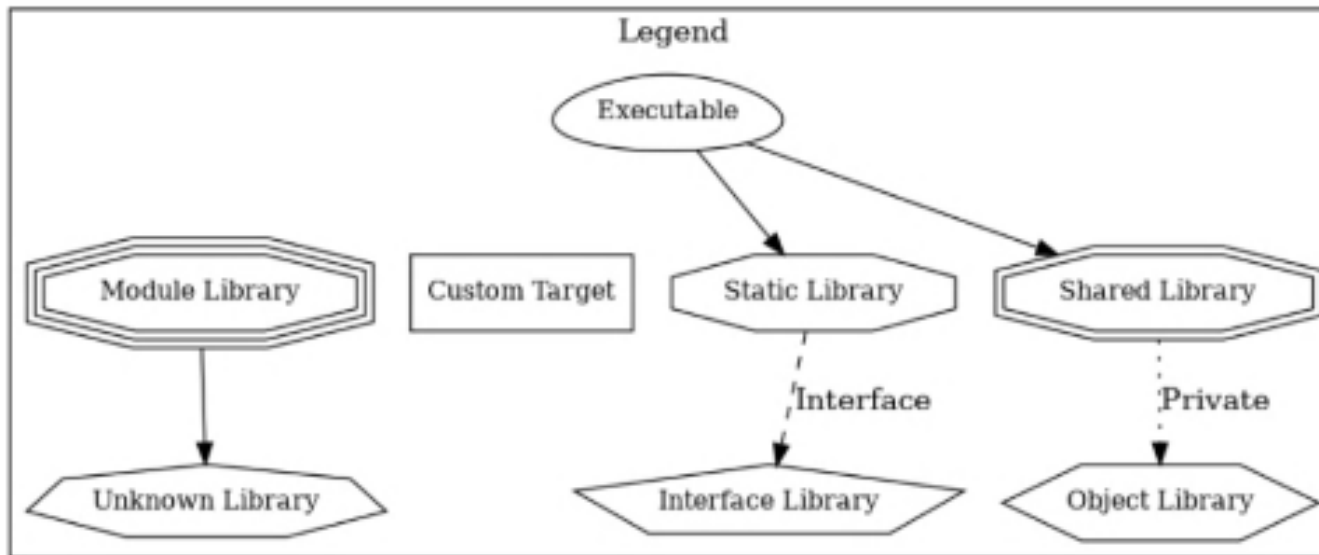
Current Process (Client Drawing)



Our Solution (Client Drawing)



Library/Dependency Structure



Testing Environment Set-Up



Cisco 140AC Ports

